MOD -03 -> Advanced Java

LUMINAR
TECHNOLAB

Java Thread

LUMINAR
TECHNOLAB

Inter-thread communication in java

Inter-thread communication or Co-operation is all about allowing synchronized threads to communicate with each other.

Cooperation (Inter-thread communication) is a mechanism in which a thread is paused running in its critical section and another thread is allowed to enter (or lock) in the same critical section to be executed. It is implemented by following methods of Object class:

wait()

notify()

notifyAll()

Object Class

1. Object Class in Java is the topmost class among all the classes in Java.
2. We can also say that the Object class in Java is the parent class for all the classes. It means that all the classes in Java are derived classes and their base class is the Object class.

3. All the classes directly or indirectly inherit from the Object class in Java.

Method	Description
public final Class getClass()	returns the Class class object of this object. The Class class can further be used to get the metadata of this class.
public int hashCode()	returns the hashcode number for this object.
public boolean equals(Object obj)	compares the given object to this object.
protected Object clone() throws CloneNotSupportedException	creates and returns the exact copy (clone) of this object.
public String toString()	returns the string representation of this object.
public final void notify()	wakes up single thread, waiting on this object's monitor.
public final void notifyAll()	wakes up all the threads, waiting on this object's monitor.

public final void wait(long timeout)throws InterruptedException	causes the current thread to wait for the specified milliseconds, until another thread notifies (invokes notify() or notifyAll() method).
public final void wait(long timeout,int nanos)throws InterruptedException	causes the current thread to wait for the specified milliseconds and nanoseconds, until another thread notifies (invokes notify() or notifyAll() method).
public final void wait()throws InterruptedException	causes the current thread to wait, until another thread notifies (invokes notify() or notifyAll() method).
protected void finalize()throws Throwable	is invoked by the garbage collector before object is being garbage collected.

1) wait() method

The wait() method causes current thread to release the lock and wait until either another thread invokes the notify() method or the notifyAll() method for this object, or a specified amount of time has elapsed.

The current thread must own this object's monitor, so it must be called from the synchronized method only otherwise it will throw exception.

- public final void wait()throws InterruptedException It waits until object is notified.
- public final void wait(long timeout)throws InterruptedException. It waits for the specified amount of time.

2) notify() method

The notify() method wakes up a single thread that is waiting on this object's monitor. If any threads are waiting on this object, one of them is chosen to be awakened.

- public final void notify()

3) notifyAll() method

Wakes up all threads that are waiting on this object's monitor.

- public final void notifyAll()

Q:

Why wait(), notify() and notifyAll() methods are defined in Object class not Thread class?

It is because they are related to lock and object has a lock.

Difference between wait and sleep?

wait()	sleep()
The wait() method releases the lock.	The sleep() method doesn't release the lock.
It is a method of Object class	It is a method of Thread class
It is the non-static method	It is the static method
It should be notified by notify() or notifyAll() methods	After the specified amount of time, sleep is completed.

Program 1

```java
class Customer{
    int amount=10000;


    synchronized void withdraw(int amount){
        System.out.println("going to withdraw...");

        if(this.amount<amount){
            System.out.println("Less balance; waiting for deposit...");
            try{
                wait();
            }catch(InterruptedException e){
                e.printStackTrace();
            }
        }

        this.amount-=amount;
        System.out.println("withdraw completed...");
    }
```

```
    synchronized void deposit(int amount){
        System.out.println("going to deposit...");
        this.amount+=amount;
        System.out.println("deposit completed... ");
        notify();
    }
}
```

```
public class Bank {
    public static void main(String args[]){

        Customer c=new Customer();

        new Thread(){
            public void run(){
                c.withdraw(15000);
            }
        }.start();

        new Thread(){
            public void run(){
                c.deposit(10000);
            }
        }.start();

    }
}
```

Console

<terminated> Bank [Java Application] C:\Program Files\Java\jre1.8.0_311\bin\javaw.exe (

```
going to withdraw...
Less balance; waiting for deposit...
going to deposit...
deposit completed...
withdraw completed...
```

Daemon Threads

Two types of threads in java: user threads and daemon threads.

User threads are high-priority threads. The JVM will wait for any user thread to complete its task before terminating it.

On the other hand, daemon threads are low-priority threads whose only role is to provide services to user threads.

Daemon threads are useful for background supporting tasks such as garbage collection, releasing memory of unused objects and removing unwanted entries from the cache.

Daemon Threads: Daemon Threads are considered as secondary threads. That means, JRE doesn't wait for the completion of a daemon thread. If the execution of all non-daemon threads gets over, JRE will terminate the program without bothering about the completion of the daemon thread.

By default, all threads are non-daemon, we can set any thread as daemon before starting it.

Scenario : We make threads in Java to do multitasking i.e. one thread ie main thread is playing video and some other thread is playing background music.Now, if the video gets over i.e. the execution of main thread finishes, then we wish to stop the music too. The JRE should not wait for the music thread to finish its execution (i.e. play the entire sound clip), so we can make the music thread as a daemon thread.

Methods for Java Daemon tThread by Thread Class

Method	Description
public void setDaemon(boolean status)	This method marks the current thread is a daemon thread or a user thread.
public final boolean isDaemon()	This method is used to determine whether or not the current thread is a daemon.If the thread is Daemon, it returns true.Otherwise, false is returned.

```
public class Demothread extends Thread {

public Demothread(String name){
     super(name);
  }

  public void run() {
      if(Thread.currentThread().isDaemon()) {
         System.out.println(getName() + " is Daemon thread");
     }

     else {
         System.out.println(getName() + " is User thread");
     }
  }
```

```
  public static void main(String[] args) {

      Demothread thread1 = new Demothread("thread1");
      Demothread thread2 = new Demothread("thread2");

      // Setting user thread thread1 to Daemon
thread1.setDaemon(true);

      // starting first 2 threads
    thread1.start();
    thread2.start();

}
}
```

Output:
thread2 is User thread
thread1 is Daemon thread

Thread Pool

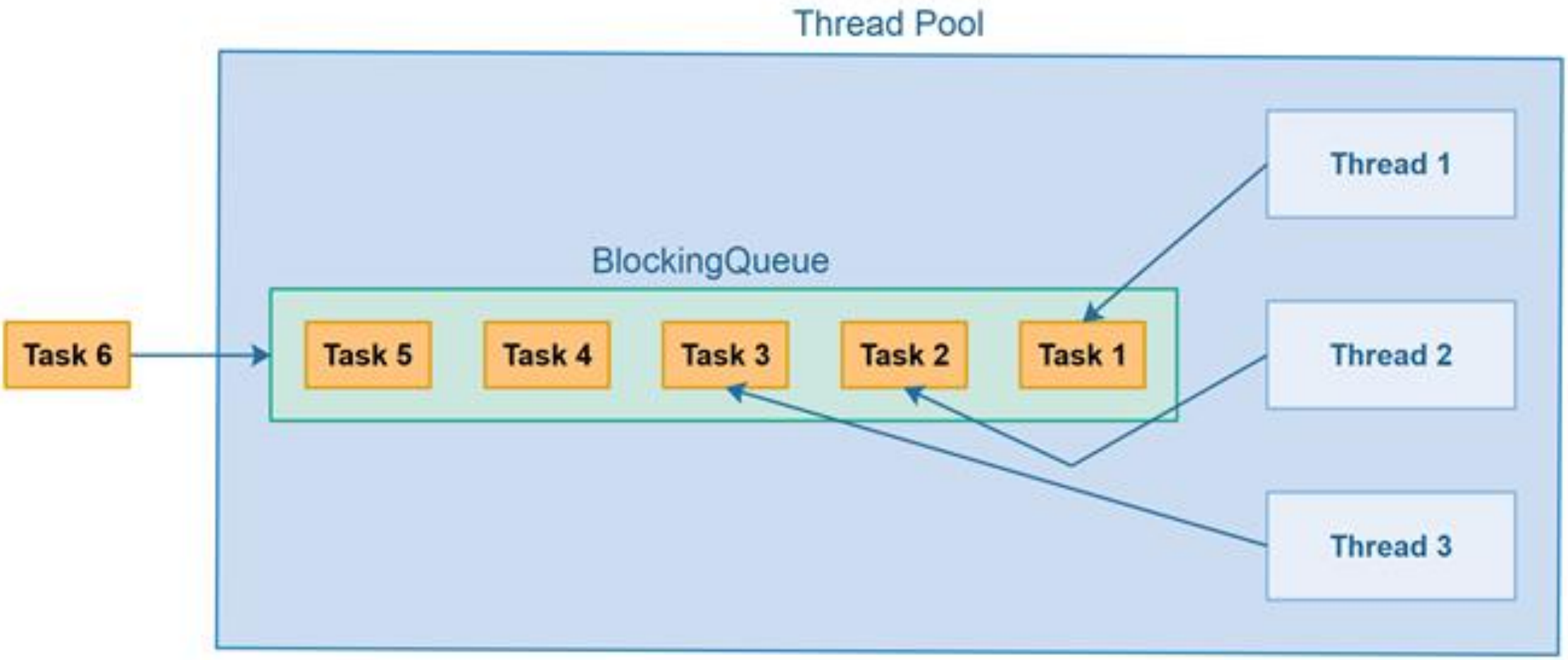
A thread pool is a pool of threads that can be "reused" to execute tasks, so that each thread may execute more than one task. A thread pool is an alternative to creating a new thread for each task you need to execute.

How a Thread Pool Works

1. Instead of starting a new thread for every task to execute concurrently, the task can be passed to a thread pool. As soon as the pool has any idle threads the task is assigned to one of them and executed.

2. Internally the tasks are inserted into a Blocking Queue which the threads in the pool are dequeuing from.

3. When a new task is inserted into the queue one of the idle threads will dequeue it successfully and execute it. The rest of the idle threads in the pool will be blocked waiting to dequeue tasks.


Thread Pool
BlockingQueue
Task 6
Task 5
Task 4
Task 3
Task 2
Task 1
Thread 1
Thread 2
Thread 3

Java comes with built in thread pools in the java.util.concurrent package, so you don't have to implement your own thread pool.

How to Create a Thread Pool in Java

The java.util.concurrent package provides several classes that can be used for this purpose, including the Executors class and the ThreadPoolExecutor class.

The java.util.concurrent package defines three executor interfaces:

1. Executor, a simple interface that supports launching new tasks.

2. ExecutorService, a subinterface of Executor, which adds features that help manage the life cycle, both of the individual tasks and of the executor itself.

3. ScheduledExecutorService, a subinterface of ExecutorService, supports future and/or periodic execution of tasks.

Concurrency Concurrency actually means that multiple tasks can be executed in an overlapping time period. One of the tasks can begin before the preceding one is completed; however, they won't be running at the same time. The CPU will adjust time slices per task and appropriately switch contexts. That's why this concept is quite complicated to implement and especially debug.

Parallelism Parallelism is the ability to execute independent tasks of a program in the same instant of time. Contrary to concurrent tasks, these tasks can run simultaneously on another processor core, another processor, or an entirely different computer that can be a distributed system. As the demand for computing speed from real-world applications increases, parallelism becomes more common and affordable.

Starvation & Deadlock

Starvation is the state where one (or more) thread is deprived of a chance to access an object. The problem can be resolved by setting the correct priorities to all the threads.

Deadlock refers to the situation where a thread is waiting for a condition, but somewhere else in the program prevented the condition from being fulfilled, thus, prevented the thread from executing.

Eg:
thread 1 is holding the lock to object A and
thread 2 is holding the lock to object B.

Thread 1 is waiting to acquire the lock to object B and
thread 2 is waiting to acquire the lock to object A. Both threads are in deadlock and cannot proceed.

How to avoid deadlocks?

Avoid Nested Locks: This is the main reason for deadlock. Deadlock mainly happens when we give locks to multiple threads. Avoid giving locks to multiple threads if you already have given to one.

Avoid Unnecessary Locks: You should lock only those members which are required. Having unnecessary locks can lead to a deadlock. As a best practice, try to reduce the need to lock things as much as you can.

Practice Questions

Create a customer-waiter-chef program in Java involves implementing a multithreaded application to simulate the interaction between customers, waiters and chefs in a restaurant.

Practice Questions

Producer – Consumer Problem

Thank you ☺ Happy learning ☺